

The Alberta Buck - Platform Architecture (DRAFT v0.1)

Perry Kundert

2026-07-03 12:00:00

Bootstrapping the BUCK monetary system requires three deliverables that share one foundation:

1. **High-resolution simulation:** 10^3 - 10^4 agents (citizens earning, living and saving; investors; businesses; governments) transacting against the real BUCK contracts and AMM pools over multi-year horizons.
2. **Live-interactive demonstrations:** browser-hosted simulations – in web sites and slide decks – where an audience member creates agents, mints BUCK_CREDIT, executes identity-signed transfers, and watches background agents rebalance real AMM pools under simulated time and price feeds.
3. **Production platforms:** JS, native, and Python libraries with which the initial identity registries, insurers, and credit unions integrate their back-office systems – signing Identities, generating BuckCredit NFTs, producing BUCK account products – and on which the first BUCK native wallets are built (Identity "fountain" management, Ethereum transaction construction and signing).

The shared foundation is (a) a backend-agnostic **ChainSession** API with expectation checking and journaling, spanning anvil, public testnets, Tenderly overlays, and in-process JS EVMs (Tevm); and (b) a Rust mathematical kernel – monetary arithmetic and the Identity/proving cryptography – compiled to WASM for JavaScript and bound natively for Python, so every platform predicts bit-identical fees, demurrage, depreciation, and proofs.

Contents

1	Decisions of record	3
2	Architecture: three layers	4
2.1	Layer 1: <code>ChainSession</code> – one testing API over every backend	4
2.2	Layer 2: the deterministic kernel – one Rust workspace, two bindings	5
2.3	Layer 3: agent platforms – Python today, JS as a peer	6
3	Repository layout and transition strategy	7
4	Phase plan	7
4.1	Phase 0: seam + scaffold [status: DONE 2026-07-03]	7
4.2	Phase 1: JS <code>ChainSession</code> + walking-skeleton agents [status: DONE 2026-07-03]	8
4.3	Phase 2: <code>buck-math</code> + bindings	8
4.4	Phase 3: <code>buck-identity</code> port	8
4.5	Phase 4: JS sim platform + live-interactive demos	9
4.6	Phase 5: scale-up	9
4.7	Phase 6: partner SDKs + wallet productization	9
5	Risks and open questions	9

1 Decisions of record

Settled 2026-07-02/03; revisit deliberately, not accidentally.

- **Population target:** 10^3 - 10^4 agents. Some "fat" (one agent driving many identities), but BUCK_CREDIT-bearing agents must stand alone. This scale does NOT require abandoning anvil: `ChainSession` over anvil is the primary sim backend; `pyrevm` (in-process revm) remains an **optional** accelerator gated by identical-vector tests against anvil.
- **No pure-Rust simulator.** Python (and later JS) orchestrate; Rust holds only the deterministic math kernels. Understandability of the sim and its agents is paramount: the chain API stays tidy, agents stay simple.
- **JS live-interactive demos are a requirement**, not a nice-to-have. Record/replay does not cut it: demos must let someone set up simulated agents, create BuckCredit, watch balances adjust, and execute transfers using real identities while background agents rebalance the real AMM pools on simulated time and prices. This implies a JS agent-simulation platform that is a **peer** of `alberta_buck/sim` – justified because (a) most researchers will never run Python, (b) most integrating organizations prefer JS, and (c) the demo wallet should be the **real** JS wallet signing real transactions.
- **The full Identity and proving stack must be available to both JS and Python** for sim and application R&D.
- **Layout:** the existing Python `alberta_buck/` keeps working throughout. New Rust, JS, and Python implementations land under `core/`, and `alberta_buck/` transitions onto them incrementally. Dependency rule: `core` never imports `alberta_buck`; `alberta_buck` may import `buck_core`.
- **Tests:** every ported/new core capability is covered in Rust, Python and JS – minimal in Rust (the kernel is small and vector-driven), primarily in Python and JS (the platforms and bindings).
- **The Solidity contracts are the mathematical specification.** Golden vectors generated by forge from the deployed-code paths are the cross-language ground truth; Rust/Python/JS suites assert byte-equality against the same vector files.

2 Architecture: three layers

Layer 3	agent platforms (scenarios, TOML experiments, demos)	alberta_buck/sim (Python) experiment harness, sweeps, eqmetrics	core/js sim (TS/JS) live-interactive browser demos
Layer 1	ChainSession API (send/call/deploy, expect=OK/REVERT, JSONL journal)	Web3Session (Python) anvil testnet Tenderly [pyrevm] _____ shared JSONL journal _____/	viem/tevm session (JS) tevm-in-browser anvil testnet
Layer 2	deterministic kernel (Rust -> WASM + Py03)	core/rust: buck-math, buck-identity, buck-wallet golden vectors from forge = the spec	

Layer numbering is historical (session API first, kernel second, platforms third); the dependency order is kernel <- session <- platforms.

2.1 Layer 1: ChainSession – one testing API over every back-end

The API is the product; execution engines plug in underneath. It is the formalization of what `alberta_buck/sim/chain.py`'s `Chain` already was, plus two ideas from the bootstrap requirements:

- **Expectations:** every state-changing operation declares whether it is expected to succeed or revert (`expect=OK` default, `expect=REVERT`). An unexpected revert raises (as today); an unexpected success logs a warning and is recorded. Expectation mismatches are first-class results, visible in the journal and countable by tests.
- **Journal:** an append-only JSONL record of every operation – tag, op, function, sender, expectation, outcome, gas, tx hash, block, error. Journals make runs auditable, diffable across backends and languages, replayable for demos, and memoizable ("the test infrastructure remembers our results"). Journaling is off by default (zero hot-path cost) and enabled per-session or via `BUCK_JOURNAL=path`.

Journal entry schema (one JSON object per line; unknown fields ignored):

field	type	meaning
i	int	1-based sequence number within the session
tag	str	caller-supplied label ("deploy:Buck", "saver3:buy")
op	str	send / deploy (calls are not journaled)
fn	str	contract function name or "constructor"
sender	str	from-address
expect	str	ok / revert
outcome	str	ok / revert
matched	bool	outcome == expectation
gas	int	gasUsed from the receipt
tx	str	transaction hash (hex)
block	int	block number
err	str	revert reason when outcome is revert

Backends, fidelity-vs-speed:

backend	runs in	role
anvil (foundry)	local process	PRIMARY sim backend; fidelity anchor
public testnets	remote	staging, partner integration
Tenderly virtual nets	remote	shared demos, partner staging
Tevm (JS, ethereumjs)	browser / node	live-interactive demos, JS sim
pyrevm (optional)	in-process	acceleration, iff vectors identical

2.2 Layer 2: the deterministic kernel – one Rust workspace, two bindings

Following the proven `holo-mutualcredit` shape (core crate -> `mutualcredit-js` wasm-bindgen package -> Holochain zomes), modernized and extended with a Python binding:

- **buck-math**: integer-only monetary arithmetic – BUCK_CREDIT declining-balance depreciation (discrete whole-year compounding, linear partial year, 100-year cap), demurrage / buckSeconds carrying apportionment, `fundingFactor()`, PID controller arithmetic (rescaled ppm), fees. No floats anywhere; U256/ppm/bp semantics matching Solidity. `no_std`-compatible core, preserving the Holochain zome option.
- **buck-identity**: BN254 (arkworks `ark-bn254`, replacing pure-Python `py_ecc`), Poseidon, Pointcheval-Sanders credentials, ElGamal, Schnorr, Chaum-Pedersen, the NIZK envelope/receipt/unilateral A1/A2 transcripts. This is simultaneously the wallet's production cryptography

and the sim's scaling bottleneck (identity registration is why `identity-cache.json` exists).

- **buck-wallet**: Identity "fountain" management, transaction construction and signing (alloy), account products. Later backed by a Holochain DHT – the same crates compile into zomes.
- **bindings/js**: wasm-bindgen + wasm-pack -> npm package (browser + Node).
- **bindings/py**: PyO3 + maturin -> wheels.

Bit-identical discipline: forge writes golden vectors (`test/vectors/math-*.json`) exercising the Solidity paths; Rust `cargo test`, Python `pytest`, and JS `node --test` all assert byte-equality against the same files. The existing Python `alberta_buck/wallet` remains the executable reference during the port, then becomes a thin shim over the PyO3 binding.

2.3 Layer 3: agent platforms – Python today, JS as a peer

Python (`alberta_buck/sim`) keeps its role: the research harness – TOML experiments, interventions, sweeps, eqmetrics, plots. It gains speed and rigor from Layers 1-2 but is not rewritten.

JS (`core/js`) grows into a peer platform in stages:

1. A **ChainSession** over viem with two transports: Tevm (in-process, browser-capable) and HTTP (anvil / testnets) – so JS agents can join the SAME anvil instance as a running Python sim (mixed-language sims), or run standalone in the browser.
2. A small, understandable agent API (mirroring the Python `setup()` / `act(day, tick)` shape) with simulated-time progression and price-feed plumbing, sharing the scenario TOML vocabulary.
3. The interactive demo layer: create agents from the page, mint Buck-Credit, transfer with REAL identities (the wasm `buck-identity` kernel proving in-browser), while whale/arb agents rebalance the AMM pools in the background.

Divergence between the two platforms is controlled by (a) both consuming the same compiled kernels for all money/identity math, (b) the shared journal format making cross-platform runs diffable, and (c) a canonical scenario whose Python-run and JS-run journals are compared in CI (Phase 4+).

3 Repository layout and transition strategy

core/	
README.md	layout, build/test commands, dependency rules
python/	buck_core package (pyproject; sessions, later Py03 re-exports)
buck_core/	session.py (Expect/Journal/Web3Session), artifacts.py
tests/	pytest suite (primary)
js/	@alberta-buck/core (ESM; journal now, session+agents next)
src/ test/	node --test suite (primary; zero deps until tevm lands)
rust/	cargo workspace: buck-math (+ buck-identity, buck-wallet, bindings/js, bindings/py as they land); minimal tests
vectors/	cross-language fixtures (journal-sample.jsonl, ...)
test/vectors/	forge-generated golden math/identity vectors (the spec)
alberta_buck/	UNCHANGED consumer; transitions module-by-module

Transition mechanics:

- Bootstrap import shim: `alberta_buck/__init__.py` prepends `core/python` to `sys.path` when running from a checkout, until `buck-core` ships as an installable wheel (Phase 2). `alberta_buck/sim/chain.py` keeps exporting `Chain` / `load_artifact` / `repo_root` forever; they become re-exports.
- Each phase flips specific `alberta_buck` internals onto `core` and states the regression gate that proves nothing moved (usually: routing smoke + canonical equilibrium vector unchanged).
- Nothing in `core` may import `alberta_buck`.

4 Phase plan

Each phase ships value standing alone, has explicit acceptance criteria, and is banked with vectors/journals before the next begins.

4.1 Phase 0: seam + scaffold [status: DONE 2026-07-03]

`core/` scaffold; Python `ChainSession` (expectation + journal) extracted from `sim/chain.py`, which becomes a delegating subclass; JS journal reader; Rust workspace seed; Make targets (`core-test{,-py,-js,-rust}`).

- *Accept:* routing smoke green on the new session; core Python/JS/Rust tests green; the same journal fixture parses identically in Python & JS.

- *Met*: routing smoke green; suites 9 py / 5 js / 3 rust; a journaled live sim run (491 ops, 153M gas, zero mismatches) summarized by the JS reader.

4.2 Phase 1: JS ChainSession + walking-skeleton agents [status: DONE 2026-07-03]

viem-based session with Tevm and anvil transports; journal writer emitting the shared schema; a whale + one arb agent in TS/JS.

- *Accept*: the JS agent pair runs (a) against a Python-deployed anvil sim (mixed-language) and (b) standalone on Tevm in Node; journals validate.
- *Met*: (a) `test_js_join_web3.py` – Python deploys the routing stack, the JS `PinWhale` snaps the real TOKEN/USDC pool along a +/- 5% path and the `RoundTripTrader` round-trips through it (incl. a DECLARED SPL revert), journal validated by the Python reader, final spot within 2% of target; (b) `onepool.tevm.test.js` – the same agents on a V3 pool built wholly in-process on Tevm (~2.7s). Tevm notes: pass `chain: null` + explicit gas with local accounts; keep deploy gas $\leq$ ~15M (block gas limit); decode revert reasons via `e.walk(ContractFunctionRevertedError)(reason or data.errorName)`.

4.3 Phase 2: buck-math + bindings

Integer kernel (depreciation, demurrage/buckSeconds, fundingFactor, PID ppm, fees); forge-generated golden vectors; wasm-pack npm package; PyO3 wheel; `alberta_buck` sim mirrors (e.g. the agents' fundingFactor) and wallet fee math flip to the binding.

- *Accept*: Rust/Python/JS byte-equal on the forge vectors; canonical equilibrium vector unchanged after the sim flips to the binding.

4.4 Phase 3: buck-identity port

arkworks BN254 + Poseidon + PS + ElGamal + Schnorr + Chaum-Pedersen + NIZK transcripts; PyO3 + wasm bindings; `alberta_buck/wallet` delegates.

- *Accept*: existing wallet vectors (`alberta_buck/wallet/vectors.py`, SNARK fixtures) reproduce bit-identically; on-chain registration in the

sim verifies; identity-cache regeneration matches; in-browser proof generation demonstrated.

4.5 Phase 4: JS sim platform + live-interactive demos

Agent API + simulated time + price feeds on Tevm-in-browser; the demo scenario (interactive agent creation, BuckCredit mint, identity-signed transfers, background AMM rebalancing); slide-deck/web embedding.

- *Accept:* the demo scenario runs wholly in-browser with real identities; a canonical mini-scenario's JS and Python journals agree.

4.6 Phase 5: scale-up

10^3 - 10^4 agent populations (extensive-margin adoption experiments from EQUILIBRIUM.md live here); throughput work (tx batching, session-level caching); OPTIONAL pyrevm backend behind the identical-vector gate.

- *Accept:* target population at acceptable wall-clock on anvil (or pyrevm with vectors proven identical); equilibrium acceptance gate still passes.

4.7 Phase 6: partner SDKs + wallet productization

buck-wallet (fountain management, tx signing) packaged for JS/Python/native; registrar/insurer/credit-union integration SDKs (same crates, thin service shims); Holochain zones consuming **buck-identity** $\neq$ **buck-wallet**.

- *Accept:* an end-to-end partner flow (sign Identity $\rightarrow$ mint BuckCredit $\rightarrow$ BUCK account product) driven purely through the SDKs on a Tenderly or testnet **ChainSession**.

5 Risks and open questions

- **Tevm fidelity/performance:** ethereumjs-EVM is slower than revm and its anvil-compatible surface is partial. Mitigation: demos are small populations; the identical-journal gate (Phase 4) catches semantic drift; anvil remains the fidelity anchor.
- **Two agent platforms drifting:** mitigated by shared kernels, shared scenario vocabulary, shared journal diffing – and by keeping agents deliberately simple (a decision of record).

- **Crypto port correctness:** mitigated by the executable Python reference, frozen fixtures, and porting transcript-by-transcript with vectors at every step. The Poseidon constants and rng-stream pinning (`vectors.py`) must be treated as immovable.
- **wasm-bindgen/PyO3 version churn:** pin toolchains in the flake; the `holo-mutualcredit` `wasm-bindgen` pin is a cautionary tale.
- **Open:** npm package naming/scope; whether `core/js` adopts TypeScript from Phase 1 (leaning yes, once `tevm/viem` land as dependencies); wheel distribution channel for partners; when to split `core/` into its own repository (not before partners need it).